

Home Network Security Lab — Full Build Guide

Author: Martin Praise

This guide is a complete, step-by-step walkthrough for building the exact SOC (Security Operations Center) home lab documented in this repository — network foundation, firewall, IDS, SIEM, and live attack simulation. Every command used during the actual build is included, in the order it was run, along with every download link needed to get the required software. Follow it top to bottom to reproduce the lab from scratch.

This guide covers Phases A–E (foundation through attack simulation). It does not include the deception-layer phase, which is being built separately.

Table of Contents

1. [Prerequisites & Downloads](#1-prerequisites--downloads)
2. [Phase A — Hypervisor & Network Foundation](#2-phase-a--hypervisor--network-foundation)
3. [Phase B — Attacker & Victim Machines](#3-phase-b--attacker--victim-machines)
4. [Phase C — Visibility: Snort IDS & Wireshark](#4-phase-c--visibility-snort-ids--wireshark)
5. [Phase D — Wazuh SIEM Deployment](#5-phase-d--wazuh-siem-deployment)
6. [Phase E — Attack Simulation & Detection Exercises](#6-phase-e--attack-simulation--detection-exercises)
7. [Final Machine/Network Reference](#7-final-machinenetwork-reference)

1. Prerequisites & Downloads

Host system used for this build: **16 GB RAM, 4 CPU cores, Linux**. Download every ISO and installer below before starting so you aren't waiting mid-build.

Software	Purpose	Download link
VMware Workstation Pro	Hypervisor (free for personal use)	https://www.vmware.com/products/workstation-pro.html
pfSense CE (ISO)	Firewall / router	https://www.pfsense.org/download/
Kali Linux (VMware pre-built image)	Attacker machine	https://www.kali.org/get-kali/#kali-virtual-machines
Ubuntu Server (ISO, 22.04/24.04 LTS)	Victim / monitored endpoint	https://ubuntu.com/download/server
Windows 10 (ISO or evaluation VM)	Victim / monitored endpoint	https://www.microsoft.com/en-us/software-download/windows10 (or Microsoft's evaluation center for a dev VM)
Wazuh OVA	SIEM (Manager + Indexer + Dashboard, single-node)	https://documentation.wazuh.com — Deployment → Virtual Machine (OVA)

Software	Purpose	Download link
Sysmon	Windows endpoint telemetry	https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon
SwiftOnSecurity Sysmon config	Community-standard Sysmon ruleset	https://github.com/SwiftOnSecurity/sysmon-config

2. Phase A — Hypervisor & Network Foundation

2.1 Install VMware Workstation Pro

```
``bash sudo apt update sudo apt install build-essential linux-headers-$(uname -r) gcc make -y chmod +x
VMware-Workstation-*.bundle sudo ./VMware-Workstation-*.bundle vmware &
```

Confirm the install:

```
``bash
vmware -v
```

If VMware's kernel modules fail to load (Could not open /dev/vmmon), diagnose and rebuild them: ```bash uname -r lsmod | grep vm sudo /usr/lib/vmware/bin/vmware-modconfig --icon-type=svg --list sudo vmware-modconfig --console --install-all`

2.2 Create the isolated internal network (do this before any VM)

1. Open **Edit → Virtual Network Editor** (click **Change Settings** for admin rights).
2. Click **Add Network**, choose **VMnet2**.
3. Set type to **Host-only**.
4. **Uncheck** "Use local DHCP service to distribute IP addresses to VMs" – pfSense will be the DHCP server, not VMware. *(Note: this was left checked initially in this build and caused a two-DHCP-server conflict later – see the fix in section 3.2. Uncheck it now to avoid that entirely.)*
5. Confirm **VMnet8** exists and is set to **NAT** – pfSense's WAN will use this for internet access.
6. Click **Save**.

2.3 Build the pfSense VM

1. Download the pfSense CE ISO (link above).
2. Create a new VM in VMware:
 - Guest OS: **FreeBSD 64-bit** (or "Other Linux 64-bit" if FreeBSD isn't listed)
 - RAM: **2048 MB**
 - Disk: **20 GB**
 - Network Adapter 1 → **NAT (VMnet8)** → this becomes WAN
 - Network Adapter 2 → **Custom → VMnet2** → this becomes LAN
3. Boot from the ISO and run through the installer (default ZFS/UFS options are fine).

Issue encountered: the installer failed with **"Netgate Installer Error – the system is NOT installed."** Diagnosed via the install log:

```
``bash
cat /tmp/install-log.txt
```

Root causes found: VM memory was only 256 MB, and the WAN adapter could not reach pfSense's package repository (pkg-static: Failed to fetch ... Timeout was reached). **Fix:** increase VM RAM to 2048 MB and confirm the host's internet connection / VMware's NAT service is running: ```bash sudo systemctl status vmware-networks 2>/dev/null ps aux | grep vmnet-natd`

2.4 First pfSense boot – assign interfaces

At the console menu, assign:

- **WAN** = the adapter connected to VMnet8 (NAT)
- **LAN** = the adapter connected to VMnet2 (Host-only)

Set the LAN IP to ``192.168.1.1/24`` and enable the DHCP server on LAN.

2.5 Access the pfSense web GUI

From any VM on VMnet2 (or via a browser once other VMs exist), confirm a DHCP lease:

```
```bash
ip a
```

Then browse to:

<https://192.168.1.1>

Default login: admin / pfsense — change the password immediately after first login.

## 2.6 Fix: VMware promiscuous mode block (needed before Phase C, set up now)

VMware Workstation blocks a guest's request for promiscuous mode by default, which later prevents Snort from seeing cross-host traffic. Fix it now on the pfSense VM:

1. Shut the VM down completely (**Power** → **Power Off**, not a guest-OS restart).
2. Edit the .vmx file:

```
```bash nano "/path/to/Pfsense Firewall.vmx"
```

3. Add (or correct) these two lines:

```
ethernet1.noPromisc = "FALSE" isolation.tools.setPromiscuous.disable = "FALSE"
```

4. On the host, grant device permissions:

```
```bash
sudo chmod a+rw /dev/vmnet2
```

5. Power the VM back on (a full power cycle is required — an in-guest reboot alone does not reload .vmx changes).

Verify promiscuous mode is actually active from inside pfSense's shell (console option 8): ```bash ifconfig em1

Look for `PROMISC` in the flags line.

---

### ## 3. Phase B – Attacker & Victim Machines

#### ### 3.1 Kali Linux (attacker)

1. Download the pre-built VMware image (`.7z`) from the Kali link above.
2. Extract it, then in VMware: **File** → **Open** the `.vmx`` file inside.
3. VM Settings: RAM **2048 MB**, CPUs **2**, Network Adapter → **Custom** → **VMnet2**.
4. Boot and confirm networking:  
```bash  
ip a

3.2 Ubuntu Server (Linux victim)

1. Download the Ubuntu Server LTS ISO (link above).
2. Create a new VM: Guest OS Ubuntu 64-bit, RAM **2048 MB**, CPU **1**, Disk **20 GB**,

Network Adapter → **Custom** → **VMnet2**.

3. Install with a **minimal server install**, enabling OpenSSH during setup.
4. Confirm networking:

```
```bash ip a
```

**\*\*Issue encountered:\*\*** VMs were intermittently getting the wrong gateway/DNS due to **\*\*two DHCP servers active on VMnet2 at once\*\*** (VMware's local DHCP + pfSense).  
**\*\*Fix:\*\*** in Virtual Network Editor, select **\*\*VMnet2\*\*** and uncheck **\*\*"Use local DHCP**

service to distribute IP addresses to VMs."\*\* pfSense then became the sole DHCP authority, and every VM began receiving clean `192.168.1.x` leases.

**\*\*Issue encountered:\*\*** `apt update` on Ubuntu Server failed repeatedly with **\*\*Temporary failure resolving 'archive.ubuntu.com'\*\*** (DNS-specific – `ping 8.8.8.8` succeeded, confirming routing worked). **\*\*Fix:\*\*** confirmed pfSense's DNS Resolver service was running (**\*\*Status → Services\*\*** in the pfSense GUI) and being correctly advertised via DHCP.

### ### 3.3 Windows 10 (Windows victim)

1. Obtain a Windows 10 ISO or evaluation VM (link above).
2. Create a new VM: Guest OS Windows 10 64-bit, RAM **\*\*4096 MB\*\***, CPU **\*\*2\*\***, Disk **\*\*60 GB\*\***, Network Adapter → **\*\*Custom → VMnet2\*\***.
3. Install normally through the setup wizard.
4. Confirm networking:  
``powershell  
ipconfig

## 3.4 Verify the whole network

From Kali, run a full ping sweep to confirm every host is present and reachable: ``bash nmap -sn 192.168.1.0/24

---

### ## 4. Phase C – Visibility: Snort IDS & Wireshark

#### ### 4.1 Install Snort on pfSense

1. pfSense GUI → **\*\*System → Package Manager → Available Packages\*\*** → search **\*\*Snort\*\*** → Install.
2. **\*\*Services → Snort → Snort Interfaces\*\*** → **\*\*+ Add\*\*** → bind a new instance to the **\*\*LAN\*\*** interface → Save.
3. Click the pencil/edit icon on that interface row → **\*\*Categories\*\*** tab → enable at least:
  - `emerging-scan.rules`
  - `emerging-exploit.rules`
  - `emerging-attack\_response.rules`
  - `emerging-web\_server.rules`
  - `emerging-web\_client.rules`
  - `emerging-policy.rules`
4. Save, then start the interface from the main Snort Interfaces list.

#### ### 4.2 Add custom Snort rules

Under the same interface's **\*\*Custom Rules\*\*** tab, add the following (one rule per line – Snort parses line by line, and `sid` values must be  $\geq 1,000,000$  for custom rules):

```
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Sweep Detected"; itype:8; threshold:type both,track by_src,count 5,seconds 10; sid:1000001; rev:1;)
alert tcp any any -> $HOME_NET 22 (msg:"Possible SSH Brute Force Attempt"; flags:S; threshold:type both,track by_src,count 5,seconds 60; sid:1000002; rev:1;)
alert tcp any any -> $HOME_NET any (msg:"Possible Nmap SYN Scan"; flags:S; threshold:type both,track by_src,count 20,seconds 5; sid:1000003; rev:1;)
alert tcp any any -> $HOME_NET 21 (msg:"FTP Login Attempt"; content:"USER"; nocase; sid:1000004; rev:1;)
alert tcp any any -> $HOME_NET 80 (msg:"Possible SQL Injection Attempt"; content:"union select"; nocase; sid:1000005; rev:1;)
```

Save, then restart the Snort interface for the rules to take effect.

**\*\*Issue encountered:\*\*** custom rules failed to save with **\*\*Fatal Error... Illegal direction specifier: \$HOME\_NET.\*\*** **\*\*Root cause:\*\*** a corrupted arrow character (`->`) introduced by copy-pasting from a rich-text source. **\*\*Fix:\*\*** re-typed the rules manually, one at a time, confirming each used the plain ASCII `->` arrow with no stray characters.

#### ### 4.3 Generate and detect a scan

From Kali:  
``bash  
sudo nmap -sS 192.168.1.102

Check pfSense **Services** → **Snort** → **Alerts** for a matching entry.

**Issue encountered:** Snort produced **zero** alerts despite confirmed live traffic between Kali and Ubuntu Server.

Diagnosed with a raw capture directly on pfSense's LAN interface: ```bash tcpdump -i em1 -n host 192.168.1.102`

This showed only ARP/broadcast traffic – not the actual unicast scan packets – confirming promiscuous mode was still not truly active at the hypervisor level (see fix in section 2.6). After applying that fix and doing a full VM power cycle, the same tcpdump command showed the real cross-host scan traffic, and Snort alerts began firing correctly.

#### ### 4.4 Manual packet capture and verification with Wireshark

1. In pfSense: **\*\*Diagnostics** → **Packet Capture**.
  - Interface: **\*\*LAN\*\***
  - Promiscuous Mode: checked
  - Untagged Filter → Host IP Address: Kali's IP (e.g. ``192.168.1.100``)
  - Protocol: ``tcp``
2. Click **\*\*Start\*\***, run the scan again from Kali, click **\*\*Stop\*\***, then **\*\*Download\*\*** the ``pcap``.
3. Open it on the host machine:

```
``bash
sudo apt install wireshark
wireshark ~/Downloads/<capture-file>.pcap
```
4. In Wireshark's filter bar, enter:  
`tcp.flags.syn==1 and tcp.flags.ack==0`

This isolates the bare SYN packets — the fingerprint of a SYN/stealth port scan.

## 5. Phase D — Wazuh SIEM Deployment

### 5.1 Deploy the Wazuh Manager VM

1. Download the Wazuh OVA (link above).
2. In VMware: **File** → **Open** the `.ova` (must be done from inside VMware, not by double-clicking the file in a file manager).
3. VM Settings: RAM **4096 MB**, CPUs **2**, Network Adapter → **Custom** → **VMnet2**.
4. Boot and let all services (manager, indexer, dashboard) initialize — this can take several minutes on first boot.
5. Find its IP and log into the dashboard:

```
``bash ip a
```

<https://<wazuh-manager-ip>>

Default admin credentials are shown on the VM's console output on first boot – record them immediately.

#### ### 5.2 Enroll the Ubuntu Server agent

Use the Wazuh dashboard's built-in wizard rather than a manual download URL:

1. Wazuh dashboard → **\*\*Endpoints** → **Deploy new agent**.
2. Package type: **\*\*DEB amd64\*\***.
3. Server address: the Wazuh Manager's IP (e.g. ``192.168.1.103``).
4. Agent name: ``UbuntuServer``.
5. Copy the generated install command (example form) and run it on Ubuntu Server via SSH:

```
``bash
```

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.6-1_amd64.deb && sudo WAZUH_MANAGER='19
```

## 6. Start the agent:

```
```bash sudo systemctl daemon-reload sudo systemctl enable wazuh-agent sudo systemctl start wazuh-agent sudo
systemctl status wazuh-agent
```

```
**Issue encountered:** a manually-typed `wget` URL for the `.deb` package returned
`403 Forbidden`. **Fix:** used the dashboard-generated command above instead, which
always matches the current package path/version.
```

```
**Issue encountered:** the agent failed to enroll – `ossec.log` showed repeated errors:
```

```
ERROR: (1208): Unable to connect to enrollment service at '[192.168.1.102]:1515'
```

```
**Root cause:** the agent's manager address was set to Ubuntu Server's own IP instead of
the Wazuh Manager's IP. **Fix:**
```

```
```bash
sudo nano /var/ossec/etc/ossec.conf
```

Correct the `<address>` field inside `<client><server>` to the Wazuh Manager's actual IP (192.168.1.103), save, then: ````bash sudo systemctl restart wazuh-agent sudo tail -f /var/ossec/logs/ossec.log`

```
5.3 Enroll the Windows 10 agent
```

Same dashboard wizard, this time selecting **MSI 32/64 bits**, and running the generated command in an **Administrator PowerShell** window on the Windows VM. Confirm:

```
```powershell
Get-Service -Name WazuhSvc
```

5.4 Install Sysmon on Windows

1. Download Sysmon (link above) and the SwiftOnSecurity config (link above) into the same folder on the Windows VM.

2. In an Administrator PowerShell, navigate into that folder first:

```
```powershell cd $env:USERPROFILE\Downloads\Sysmon
```

3. Install:

```
```powershell
.\Sysmon64.exe -accepteula -i sysmonconfig-export.xml
```

4. Confirm it's running:

```
```powershell Get-Service -Name Sysmon64
```

```
Issue encountered: the install command initially failed with ".\Sysmon64.exe is not
recognized" because PowerShell was still in C:\Windows\system32, not the folder
containing the extracted Sysmon files. **Fix:** cd into the correct folder first (as
above) before running the install command.
```

```
5.5 Feed Sysmon events into the Wazuh agent
```

```
```powershell
notepad "C:\Program Files (x86)\ossec-agent\ossec.conf"
```

Add a new `<localfile>` block (leave existing ones untouched): ````xml <localfile>`

```
<location>Microsoft-Windows-Sysmon/Operational</location> <log_format>eventchannel</log_format> </localfile>
```

Save, then restart the agent:

```
```powershell
Restart-Service -Name WazuhSvc
```

## 5.6 Configure File Integrity Monitoring (FIM)

**Ubuntu Server** — add realtime monitoring to the existing `<syscheck>` block: ````bash sudo nano
/var/ossec/etc/ossec.conf`

```
```xml
```

```
<syscheck>
...
<directories realtime="yes">/etc</directories>
</syscheck>
```

```
```bash sudo systemctl restart wazuh-agent
```

```
Test:
```bash
sudo touch /etc/testfile.txt
sudo rm /etc/testfile.txt
```

Check the **File Integrity Monitoring** module in the Wazuh dashboard for the resulting alert.

Windows 10 — the default FIM scope did not include C:\Windows\Temp, so it was added explicitly in the same way (edit ossec.conf, add <directories realtime="yes">C:\Windows\Temp</directories> inside <syscheck>, restart the WazuhSvc service). Test: ``powershell New-Item -Path "C:\Windows\Temp\run.exe" -ItemType File

```
### 5.7 Monitor Ubuntu's SSH authentication log (used later in Phase E)
```

```
Confirm the log exists and is active:
```bash
ls -la /var/log/auth.log
sudo tail -20 /var/log/auth.log
```

Add it as a monitored source: ``bash sudo nano /var/ossec/etc/ossec.conf

```
```xml
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/auth.log</location>
</localfile>
```

```
```bash sudo systemctl restart wazuh-agent
```

```
5.8 Forward pfSense logs to Wazuh via syslog
```

```
On pfSense: **Status → System Logs → Settings**:
```

- Enable Remote Logging
- Source Address: LAN
- Remote log servers (first field only): `192.168.1.103:514`
- Remote Syslog Contents: check **System Events** and **Firewall Events**
- Save

```
Issue encountered: the form rejected saving with "A valid IP address/hostname or IP/hostname:port must be specified for remote syslog server #2." **Root cause:** the IP and port had been entered into two separate server slots instead of one combined `IP:port` value. **Fix:** combined them into the first field (`192.168.1.103:514`) and left the second field blank.
```

```
On the Wazuh Manager, add a syslog listener:
```bash
sudo cp /var/ossec/etc/ossec.conf /var/ossec/etc/ossec.conf.bak
sudo nano /var/ossec/etc/ossec.conf
```

```
```xml <remote> <connection>syslog</connection> <port>514</port> <protocol>udp</protocol>
<allowed-ips>192.168.1.1</allowed-ips> </remote>
```

```
```bash
sudo systemctl restart wazuh-manager
sudo netstat -ulnp | grep 514
```

To verify delivery end-to-end, enable full event archiving temporarily: ``xml <global> <logall>yes</logall> <logall_json>yes</logall_json> </global>

```
```bash
sudo systemctl restart wazuh-manager
sudo tail -f /var/ossec/logs/archives/archives.log
```

Generate traffic (e.g. an Nmap scan from Kali) and confirm entries appear: ```bash sudo grep "192.168.1.1" /var/ossec/logs/archives/archives.log`

**\*\*Known limitation:\*\*** pfSense's Snort package logs its own alerts to a local file rather than through this general syslog forwarder, so Snort-specific alerts do not appear in the Wazuh dashboard through this pipeline – only general pfSense system/firewall events do. This was confirmed and documented rather than worked around.

#### ### 5.9 Investigating an alert burst (verification exercise)

An unexpected burst of ``rootcheck`` alerts (rule 510) appeared in Threat Hunting. Rather than assume a fault, this was verified directly:

```
``bash
sudo grep -i "starting rootcheck" /var/ossec/logs/ossec.log
sudo systemctl status wazuh-manager
free -h
top -bn1 | head -20
```

This confirmed the burst was expected scan-on-start behavior tied to known agent/manager restarts performed during configuration — not a bug, and no fix was required.

## 6. Phase E — Attack Simulation & Detection Exercises

### 6.1 SSH brute force (MITRE ATT&CK T1110)

From Kali: ```bash hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt -t 4 ssh://192.168.1.102`

(If the wordlist is compressed: ``sudo gunzip /usr/share/wordlists/rockyou.txt.gz`` first.)

Check:

- pfSense **\*\*Services → Snort → Alerts\*\*** for the brute-force signature
- Wazuh **\*\*Threat Hunting\*\***, searching ``sshd`` or ``agent.name: UbuntuServer`` – expect individual failed-login alerts (rule 5710) escalating into correlation alerts such as "User missed the password more than one time" (rule 2502) and "Maximum authentication attempts exceeded" (rule 5758)

#### ### 6.2 Vulnerability scanning (MITRE ATT&CK T1595)

```
``bash
nmap -sV --script vuln 192.168.1.102
```

Review the listed CVEs and their CVSS scores (0–10 scale; 9.0+ is Critical).

### 6.3 Exploit research (MITRE ATT&CK T1190)

```
``bash searchsploit openssh
```

Compare the version numbers in the results against the actual installed OpenSSH version (from the Nmap output above) to determine whether any public exploit genuinely applies – do not assume a listed CVE has a working exploit without checking this.

#### ### 6.4 Investigative pivoting in Wazuh

In Wazuh's Threat Hunting search bar, pivot on the attacker's IP across all agents to reconstruct what it touched:

srcip: "192.168.1.100"

---

#### ## 7. Final Machine/Network Reference

Machine	Role	RAM	Disk	vCPU	IP Address
pfSense Firewall	Router / Firewall	2 GB	20 GB	1	LAN: `192.168.1.1`
Kali Linux	Attacker	2 GB	40 GB	2	`192.168.1.100`



```
| Ubuntu Server | Victim / Wazuh Agent | 2 GB | 20 GB | 1 | `192.168.1.102` |
| Wazuh Manager | SIEM (Manager + Indexer + Dashboard) | 4 GB | 50 GB | 2 | `192.168.1.103` |
| Windows 10 | Victim / Wazuh Agent + Sysmon | 4 GB | 60 GB | 2 | `192.168.1.104` |
```

Networks:

- **VMnet2** – Host-only, internal LAN for every VM (DHCP served by pfSense only)
- **VMnet8** – NAT, used only by pfSense's WAN interface for internet access

---

\*Guide written by Martin Praise, documenting the exact build steps for the Home Network Security Lab project (Phases A-E).\*